Retrieval-Augmented Generation for Large Language Models: A Comprehensive Survey of Architectures, Mechanisms, and Frontiers

Abstract

The rapid evolution of Large Language Models (LLMs) has revolutionized natural language processing, yet these models remain constrained by the static nature of their parametric knowledge, leading to hallucinations, outdated information, and limited domain specificity. Retrieval-Augmented Generation (RAG) has emerged as a pivotal paradigm to address these limitations by integrating non-parametric external knowledge sources into the generation process. This survey provides a comprehensive analysis of the RAG landscape, synthesizing insights from 300 recent academic studies. We propose a unified taxonomy categorizing RAG systems based on retrieval granularity, integration depth, architectural modularity, and adaptation strategies. We systematically examine methodological advancements ranging from naive retrieve-then-read pipelines to sophisticated agentic frameworks employing iterative planning, graph-based reasoning, and parametric knowledge injection. Furthermore, we explore critical challenges including retrieval noise, knowledge conflicts, safety vulnerabilities, and evaluation bottlenecks. By synthesizing empirical evidence across diverse domains—from biomedical reasoning to code generation and legal analysis—this paper delineates the current state-of-the-art, identifies persistent gaps, and outlines future research directions for building robust, trustworthy, and efficient retrieval-augmented systems.

1. Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities in generative tasks, yet their reliance on fixed parametric memory imposes inherent constraints regarding factual accuracy, temporal relevance, and domain adaptability. The phenomenon of "hallucination," where models generate plausible but factually incorrect content, remains a significant barrier to deployment in high-stakes environments [1]. Furthermore, the cost of retraining massive models to incorporate new knowledge renders purely parametric updates impractical for dynamic domains [2]. Retrieval-Augmented Generation (RAG) addresses these challenges by decoupling knowledge storage from model parameters, allowing LLMs to access external corpora, knowledge graphs, and databases at inference time [3].

The evolution of RAG has progressed from simple concatenation of retrieved documents to complex, multi-stage orchestration frameworks. Early approaches, such as REALM, introduced the concept of joint pre-training of retrievers and encoders using masked language modeling signals to optimize retrieval for downstream generation tasks [4]. Subsequent works like Atlas and RETRO demonstrated that scaling retrieval to trillions of tokens could match or exceed the performance of significantly larger parametric models, establishing the efficacy of semi-parametric architectures [5, 6]. However, the field has since expanded beyond static retrieval, incorporating adaptive mechanisms that dynamically determine *when* to retrieve, *what* to retrieve, and *how* to fuse external evidence with internal knowledge [7, 8].

Current research emphasizes the modularity and adaptability of RAG systems. Modern frameworks decompose the pipeline into distinct phases: pre-retrieval query optimization, iterative retrieval-generation loops, post-retrieval filtering, and advanced fusion strategies [9, 10]. Innovations such as Agentic RAG treat retrieval as a sequential decision process, enabling models to plan, reflect, and execute multi-step tool use [11]. Simultaneously, parametric approaches like P-RAG and Knowledge

Cards explore injecting retrieved knowledge directly into model weights via low-rank adapters or specialized modules, blurring the line between retrieval and fine-tuning [12, 13].

Despite these advances, significant challenges persist. The "lost-in-the-middle" phenomenon degrades performance when context windows are overloaded with irrelevant information [14]. Knowledge conflicts between parametric memory and retrieved documents can lead to inconsistent outputs [15]. Moreover, RAG systems introduce new attack surfaces, including prompt injection and data poisoning, while raising concerns about attribution and copyright [16, 17]. This survey synthesizes the extensive literature on RAG, organizing methods into coherent families, evaluating their empirical performance across benchmarks, and discussing the open problems that define the next frontier of knowledge-augmented AI.

2. Method

The methodological landscape of Retrieval-Augmented Generation is vast and rapidly evolving. To provide a structured analysis, we categorize existing approaches based on the depth of integration between the retriever and generator, the strategy for retrieval triggering, the structure of the knowledge base, and the mechanisms for adaptation and optimization.

2.1. Architectural Paradigms and Integration Strategies

The fundamental architecture of a RAG system is defined by how external knowledge is incorporated into the language modeling process. We identify three primary integration depths: input-layer augmentation, intermediate-layer fusion, and output-layer interpolation.

2.1.1. Input-Layer Augmentation (Naive and Advanced RAG) The most prevalent architecture involves retrieving documents and concatenating them with the user query to form an extended prompt. This "Naive RAG" approach leverages the LLM's in-context learning capabilities without modifying model weights [10]. While simple, this method is susceptible to context window limits and noise. To mitigate this, advanced techniques employ query rewriting, expansion, and decomposition. For instance, RAG-Fusion generates multiple diverse queries from a single input and aggregates results using Reciprocal Rank Fusion (RRF), significantly improving retrieval coverage [18]. Similarly, HyDE (Hypothetical Document Embeddings) and related methods generate hypothetical answers to guide retrieval, bridging the semantic gap between query and document [19].

Structural fidelity in input augmentation is also critical. DOS RAG demonstrates that preserving the original sequential order of retrieved chunks, rather than sorting solely by similarity, maintains narrative coherence and boosts performance on long-context tasks [20]. Furthermore, chunking strategies have evolved from fixed-size splitting to semantic and structure-aware methods. W-RAC reframes chunking as a planning problem, using LLMs to generate ID-based plans for web documents, reducing token usage by over 84% while maintaining quality [21]. In academic domains, PT-RAG utilizes the native hierarchy of papers to construct "PaperTrees," enabling path-guided retrieval that aligns with the document's logical flow [22].

2.1.2. Intermediate-Layer Fusion and Cross-Attention Deeper integration involves fusing retrieved information within the transformer layers, typically via cross-attention mechanisms. The RETRO architecture exemplifies this by retrieving chunks of text and attending to them via a dedicated cross-attention module in the decoder, decoupling memory capacity from parameter

count [6]. This approach allows models to access trillions of tokens of external knowledge efficiently. Extensions like RETRO++ optimize this by injecting the top-1 most relevant evidence directly into the context, improving factual accuracy without compromising fluency [23].

Alternative fusion mechanisms include memory-augmented networks. MLP Memory compresses non-parametric kNN datastores into differentiable MLP weights, imitating retriever distributions to reduce inference latency while retaining the benefits of external memory [24]. Similarly, Extended Mind Transformers integrate top-k key-value retrieval from external caches directly into standard decoder layers using relative positional encodings, enabling pre-trained models to attend to external memory without fine-tuning [25]. These methods highlight a trend towards tighter coupling where retrieval becomes an intrinsic part of the forward pass rather than a preprocessing step.

2.1.3. Output-Layer Interpolation and Ensemble Methods Output-layer strategies combine predictions from multiple sources. REPLUG treats the LM as a frozen black box, retrieving k documents and running k parallel forward passes, then ensembling the output token probabilities weighted by retrieval similarity [26]. This bypasses context length constraints and leverages the robustness of ensemble inference. GenKI adopts a three-stage pipeline where retrieved knowledge is integrated via a hybrid loss function before controllable generation, preventing distributional conflicts [27]. Such approaches are particularly effective when the generator needs to balance conflicting evidence or when the retrieval signal is noisy.

2.2. Adaptive and Dynamic Retrieval Mechanisms

A critical limitation of static RAG is the indiscriminate retrieval for every query, which introduces noise and latency. Adaptive retrieval mechanisms empower models to decide *when* and *what* to retrieve based on internal confidence or task complexity.

2.2.1. Confidence-Based and Self-Knowledge Triggering Several methods leverage the LLM's own uncertainty to trigger retrieval. SKR (Self-Knowledge guided Retrieval) partitions training data to teach the model to recognize questions it cannot answer parametrically, using performance variance as a signal for retrieval necessity [7]. Similarly, EI-ARAG utilizes pre-trained token embeddings from early layers to infer intrinsic knowledge distributions, predicting retrieval needs without extra inference calls [28]. Thrust employs geometric clustering of hidden states to quantify instance-level knowledgeability, gating retrieval based on the distance of query representations from known clusters [29].

More sophisticated approaches involve explicit control tokens. ADAPT-LLM trains models to emit a special $\langle \text{RET} \rangle$ token when external knowledge is required, learning this behavior through error analysis on training data [30]. UNIWEB extends this to web search, using output entropy estimates to trigger Google Search queries only when the model is uncertain, thereby preventing noisy augmentation [31]. These methods collectively demonstrate that adaptive triggering can match the accuracy of "always-retrieve" baselines while significantly reducing computational costs [32].

2.2.2. Iterative and Recursive Retrieval For complex multi-hop reasoning, single-step retrieval is often insufficient. Iterative frameworks interleave retrieval and generation steps. ITER-RETGEN executes multiple rounds where the output of one generation step informs the retrieval query of the next, effectively bridging semantic gaps between the initial query and supporting documents [33]. Auto-RAG models this process as a multi-turn dialogue between the LLM and

the retriever, employing reasoning-based planning to identify knowledge gaps and refine queries autonomously [34].

Recursive approaches like RAPTOR construct hierarchical summary trees, retrieving at multiple levels of abstraction to handle long documents and complex dependencies [35]. Question Decomposition (QD-RAG) breaks complex queries into sub-queries, retrieves evidence for each, and reranks the merged pool before generation, expanding evidence coverage significantly [36]. Plan*RAG externalizes this reasoning by constructing a Directed Acyclic Graph (DAG) of atomic subqueries, enabling systematic path exploration and parallel execution [37]. These iterative and recursive strategies are essential for tasks requiring the synthesis of information from disparate sources.

2.3. Knowledge Structuring and Graph-Based Retrieval

Moving beyond flat vector stores, structured knowledge representations like Knowledge Graphs (KGs) and graphs of documents offer superior reasoning capabilities, particularly for multi-hop queries.

2.3.1. Knowledge Graph Augmentation Integrating KGs into RAG provides explicit entity relationships that facilitate logical deduction. KG-RAG constructs domain-specific graphs (e.g., in telecommunications) and retrieves ontology-aligned triples, which are verbalized into declarative sentences for the LLM [38, 39]. This structured grounding reduces hallucinations compared to raw text chunks. OG-RAG maps documents to ontology triples forming factual blocks within a hypergraph, optimizing retrieval to cover minimal hyperedges relevant to the query, thereby preserving complex entity relationships [40].

In scientific domains, ATLANTIC fuses semantic embeddings with structural embeddings from a heterogeneous document graph (co-citation, co-topic), improving retrieval relevance and faithfulness in interdisciplinary science [41]. GraphRAG surveys categorize these approaches into index-based (graphs linking chunks), knowledge-based (graphs storing facts), and hybrid paradigms, noting that graph structures enable explicit modeling of hierarchies and multi-hop reasoning [42].

2.3.2. Generative and Internalized Retrieval An emerging direction involves internalizing the retrieval index within the model parameters or generating identifiers directly. Self-Retrieval unifies indexing, retrieval, and reranking within a single LLM, using constrained decoding over a corpus trie to generate exact document titles and passages [43]. Similarly, MindRef mimics human memory by using Trie-constrained decoding to generate valid Wikipedia titles, narrowing the search space before employing an FM-index for precise localization [44].

CorpusLM takes this further by employing a ranking-oriented DocID list generation strategy, learning to generate lists of document identifiers rather than single pairs, which improves retrieval robustness [45]. These "generative retrieval" methods eliminate the need for external approximate nearest neighbor search, though they face challenges in scaling to dynamic corpora without retraining [46].

2.4. Parametric Injection and Memory Adaptation

While traditional RAG keeps knowledge external, parametric approaches seek to inject retrieved information directly into the model's weights or specialized memory modules, offering a middle ground between retrieval and fine-tuning.

2.4.1. Parameter-Efficient Fine-Tuning with Retrieval Parametric RAG (P-RAG) transforms external documents into low-rank matrix updates (ΔW) for the LLM's feed-forward networks via offline training. During inference, the model retrieves relevant documents and activates the corresponding adapters, enabling deeper knowledge integration than context appending [12]. Knowledge Cards employ small, domain-specific LMs as plug-in modules that generate background documents for a black-box general LLM, effectively serving as updatable parametric knowledge repositories [13].

Fine-tuning strategies also play a role in robustness. Finetune-RAG trains LLMs on pairs of correct and fictitious document chunks, forcing the model to learn to ignore misleading context, thereby improving resistance to hallucination [47]. Conversely, studies suggest that naive fine-tuning on small domain datasets can degrade RAG performance compared to base models, highlighting the need for careful curriculum design [48]. CL-RAG addresses this by stratifying training data by difficulty (easy, common, hard), progressively exposing the generator to noise to enhance robustness [49].

2.4.2. Long-Term and Episodic Memory Systems For agentic applications, maintaining long-term memory is crucial. MemLong partitions input into chunks, caching key-value pairs in an external bank and retrieving them via frozen lower layers to maintain distributional consistency [50]. HyMem implements a dual-granularity architecture, storing event-level summaries for routine queries and raw text for complex reasoning, dynamically switching based on query complexity [51].

Advanced memory systems like AnchorMem decouple atomic fact retrieval from immutable raw context storage, constructing Associative Event Graphs to bind fragmented facts without information dilution [52]. MemInsight autonomously mines semantic attributes from dialogue history to enrich memory representation, filtering irrelevant history while retaining key insights [53]. These systems move beyond simple retrieval towards cognitive architectures that manage consolidation, forgetting, and reconstruction of memory [54, 55].

2.5. Multi-Agent and Tool-Use Frameworks

The convergence of RAG with agentic workflows has led to systems where multiple specialized agents collaborate to retrieve, verify, and synthesize information.

2.5.1. Agentic Orchestration and Planning Agentic RAG formalizes the retrieval loop as a Partially Observable Markov Decision Process (POMDP), distinguishing it from active RAG by employing policy-driven multi-step tool use [11]. MASS-RAG decomposes the pipeline into parallel agents (Summarizer, Extractor, Reasoner) that distill evidence from different perspectives before synthesis, capturing non-overlapping factual subsets [56]. MAIN-RAG employs a three-agent sequential filtering process (Predictor, Judge, Final-Predictor) to statistically separate relevant documents from noise using log-prob differences [57].

In complex domains, multi-agent systems coordinate specialized tasks. MAG-RAG uses distinct agents for extraction, terminology conversion, and optimization modeling in sensor array signal processing, leveraging a hierarchical graph structure for precise retrieval [58]. Similarly, GenAI-powered frameworks for urban mobility integrate LLM agents for orchestration with retrieval agents for fetching dynamic traffic data, enabling real-time simulation and decision-making [59].

2.5.2. Tool Learning and Function Calling RAG systems increasingly integrate with external tools beyond simple text search. LLM-Embedder unifies diverse retrieval tasks (knowledge, memory, tools) into a single embedding backbone optimized via knowledge distillation from LLM generation rewards [60]. In code generation, CONAN aligns code and documentation embeddings via contrastive learning, enabling structure-aware retrieval for code completion [61]. For hardware design, TimelyHLS integrates RAG with iterative synthesis feedback, retrieving FPGA-specific pragma templates to resolve timing violations [62]. These applications demonstrate RAG's versatility in grounding LLMs in executable tools and structured domain knowledge.

2.6. Domain-Specific Applications and Specialized Architectures

RAG has been adapted to numerous specialized domains, each presenting unique challenges and requiring tailored solutions.

2.6.1. Biomedical and Healthcare In healthcare, accuracy is paramount. RAG-LLM pipelines for preoperative medicine retrieve guideline chunks to achieve performance non-inferior to human experts [63]. LaB-RAG enhances radiology report generation by extracting image embeddings and retrieving similar cases based on CheXpert labels, bridging vision-language modalities [64]. For disease phenotyping, RAG-MapReduce segments clinical notes and aggregates binary queries via max aggregation, outperforming structured baselines [65]. However, safety remains a concern; RAG LLMs can become less safe if retrieved documents encourage bypassing safety training, necessitating rigorous evaluation [17].

2.6.2. Legal and Regulatory Compliance Legal RAG systems must handle long, complex documents and strict jurisdictional constraints. LegalBench-RAG introduces a trace-back methodology to map answers to original corpus spans, emphasizing the need for precise chunking strategies like Recursive Character Text Splitting [66]. TraCR AI implements state-wise indexing to optimize retrieval for multi-jurisdiction queries in transportation cybersecurity, eliminating hallucinations by grounding responses in exact statutory codes [67]. AQgR transforms unstructured legal judgments into structured JSON summaries to guide retrieval, significantly improving Mean Average Precision [68].

2.6.3. Code and Software Engineering In software engineering, RAG aids in code completion, review, and vulnerability detection. Retrieval-augmented code completion using Jaccard similarity outperforms complex architectures for local projects by leveraging simple token overlap [69]. For vulnerability detection, ParaVul parallelizes a fine-tuned LLM with a hybrid RAG system, fusing outputs via a meta-learning gate to detect smart contract vulnerabilities with high F1 scores [70]. RAG also enhances test generation, where API-level retrieval of documentation and GitHub issues improves line coverage compared to zero-shot baselines [71].

2.6.4. Scientific and Technical Domains Scientific RAG systems handle heterogeneous data and complex reasoning. GhostWriter integrates ontology-based term enrichment with vector embeddings to improve recall in academic paper retrieval [72]. In materials science, RAG grounds LLM generation in verified databases like the Materials Project, reducing factual errors in crystalline material discovery [73]. For high-speed train advisory systems, IDAS-LLM combines fine-tuning with RAG on railway syllabi to improve terminology accuracy and regulation recall [74].

2.7. Evaluation, Robustness, and Safety

As RAG systems proliferate, robust evaluation frameworks and safety mechanisms have become critical areas of research.

2.7.1. Evaluation Frameworks and Metrics Traditional metrics like BLEU and ROUGE are often insufficient for RAG. New benchmarks like MultiHop-RAG introduce "Null queries" to explicitly evaluate hallucination resistance in multi-hop scenarios [75]. RGB constructs dynamic testbeds with varying noise ratios to stress-test robustness, revealing that LLMs struggle with negative rejection and information integration under noise [76]. THELMA proposes a reference-free evaluation framework that decomposes inputs into atomic claims to quantify precision-coverage trade-offs [77]. Additionally, LLM-as-a-Judge approaches like AutoNuggetizer automate fact extraction and assignment, correlating strongly with manual methods at the run level [78].

2.7.2. Robustness Against Noise and Conflicts Retrieval noise and knowledge conflicts are major failure modes. ASTUTE RAG employs iterative source-aware consolidation to merge internal and external knowledge, filtering inconsistencies based on constitutional principles [15]. Corrective RAG (CRAG) uses a lightweight evaluator to assess document relevance, triggering corrective actions like web search or decomposition when retrieval is deemed incorrect or ambiguous [79]. SKILL-RAG leverages reinforcement learning to elicit model self-knowledge, filtering context based on entropy-weighted advantages to reduce hallucination [80].

2.7.3. Safety, Security, and Attribution RAG introduces new vulnerabilities. CPA-RAG demonstrates covert poisoning attacks where malicious text is optimized to manipulate both retrieval and generation objectives [16]. Safety analysis reveals that even safe models can produce unsafe outputs in RAG settings if retrieved documents contain repurposed safe information, highlighting the need for end-to-end safety alignment [17]. Attribution bias is another concern; models exhibit systematic bias towards documents labeled with specific authorship metadata, necessitating counterfactual evaluation protocols to ensure trustworthiness [81]. Watermarking techniques like RAG-WM embed semantic knowledge tuples to protect IP in RAG outputs [82].

3. Challenges and Outlook

Despite significant progress, Retrieval-Augmented Generation faces several persistent challenges that hinder its widespread adoption and reliability.

3.1. The Retrieval-Generation Gap and Knowledge Conflicts

A fundamental tension exists between the retriever's objective (semantic similarity) and the generator's needs (factual utility). Retrievers often miss essential low-similarity statements, constituting a primary bottleneck for multi-hop reasoning [83]. Furthermore, conflicts between parametric memory and retrieved documents can lead to inconsistent or hallucinated outputs. While methods like ASTUTE RAG and CRAG attempt to resolve these conflicts, a unified theoretical framework for modeling and mitigating knowledge dissonance remains elusive [15, 79]. Future research must focus on joint optimization objectives that align retriever incentives with generator faithfulness, potentially through end-to-end differentiable retrieval or reinforcement learning from generation feedback [84, 85].

3.2. Scalability and Efficiency

The computational overhead of RAG is substantial. Iterative retrieval, multi-agent orchestration, and large context windows incur significant latency and token costs [9, 86]. While parametric injection methods like P-RAG and memory compression techniques like MemLong offer pathways to efficiency, they often involve expensive offline training or complex architectural changes [12, 50]. The trade-off between retrieval depth and inference speed remains a critical design constraint. Future directions include developing lightweight retrievers, optimizing cache mechanisms, and exploring speculative decoding strategies that verify retrieved context asynchronously [87, 88].

3.3. Evaluation Bottlenecks and Benchmarking

Current evaluation metrics often fail to capture the nuances of RAG performance, such as faithfulness, attribution, and reasoning depth. Automated metrics like BERTScore may not correlate with human judgment on complex reasoning tasks [89]. The reliance on LLM-as-a-Judge introduces reproducibility issues due to prompt sensitivity and model bias [90, 11]. There is an urgent need for standardized, multi-dimensional benchmarks that evaluate not just answer accuracy but also the quality of retrieval, the robustness to noise, and the system's ability to abstain when knowledge is insufficient [76, 66].

3.4. Safety, Privacy, and Ethical Concerns

RAG systems amplify risks related to data privacy and content safety. Retrieving from uncurated web sources or user data can expose models to toxic content, prompt injection attacks, and private information leakage [17, 91]. The "Spiral of Silence" effect, where retrieval models bias towards LLM-generated content, threatens the diversity and authenticity of information ecosystems [92]. Moreover, copyright and attribution issues arise when models generate content based on retrieved proprietary data. Future work must prioritize developing robust guardrails, differential privacy mechanisms for retrieval, and transparent attribution frameworks that trace generated claims back to their sources [82, 81].

3.5. Future Directions

The future of RAG lies in the convergence of retrieval, reasoning, and memory. We anticipate a shift towards **Agentic RAG**, where models autonomously plan, execute, and verify multi-step retrieval strategies in dynamic environments [11, 93]. **Multimodal RAG** will expand beyond text to integrate images, audio, and video, requiring novel fusion mechanisms for cross-modal reasoning [94, 95]. **Personalized RAG** will leverage user-specific memory banks to tailor responses while preserving privacy through techniques like federated learning and local execution [96, 97]. Finally, **Neuro-Symbolic RAG** may emerge, combining the flexibility of LLMs with the rigor of symbolic reasoning over structured knowledge graphs to achieve verifiable and explainable AI [98, 99].

4. Conclusion

Retrieval-Augmented Generation has established itself as a cornerstone of modern AI, effectively bridging the gap between the static knowledge of parametric models and the dynamic, vast expanse of external information. This survey has synthesized a diverse array of methodologies, from naive input augmentation to sophisticated agentic frameworks and parametric injection techniques. We have highlighted how innovations in adaptive triggering, graph-based reasoning, and multi-agent

orchestration are pushing the boundaries of what LLMs can achieve in terms of factual accuracy and reasoning depth.

However, the path forward is not without obstacles. Challenges related to retrieval noise, knowledge conflicts, computational efficiency, and safety vulnerabilities remain significant. Addressing these issues requires a holistic approach that integrates advances in retrieval algorithms, model architecture, evaluation frameworks, and ethical safeguards. As the field matures, the distinction between retrieval and generation will likely continue to blur, giving rise to unified cognitive architectures capable of autonomous knowledge seeking, verification, and synthesis. By fostering interdisciplinary collaboration and rigorous benchmarking, the research community can unlock the full potential of RAG, enabling trustworthy and capable AI systems for complex real-world applications.

References

[1] Wenqi Fan, Yujuan Ding, Liangbo Ning. (2024). A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models. arXiv:2405.06211.

[2] Shangyu Wu, Ying Xiong, Yufei Cui. (2024). Retrieval-Augmented Generation for Natural Language Processing: A Survey. arXiv:2407.13193.

[3] Akari Asai, Zexuan Zhong, Danqi Chen. (2024). Reliable, Adaptable, and Attributable Language Models with Retrieval. arXiv:2403.03187.

[4] Kelvin Guu, Kenton Lee, Zora Tung. (2020). REALM: Retrieval-Augmented Language Model Pre-Training. arXiv:2002.08909.

[5] Gautier Izacard, Patrick Lewis, Maria Lomeli. (2022). Atlas: Few-shot Learning with Retrieval Augmented Language Models. arXiv:2208.03299.

[6] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann. (2022). Improving language models by retrieving from trillions of tokens. arXiv:2112.04426.

[7] Yile Wang, Peng Li, Maosong Sun. (2023). Self-Knowledge Guided Retrieval Augmentation for Large Language Models. arXiv:2310.05002.

[8] Weihang Su, Qingyao Ai, Jingtao Zhan. (2025). Dynamic and Parametric Retrieval-Augmented Generation. arXiv:2506.06704.

[9] Yizheng Huang, Jimmy X. Huang. (2024). The Survey of Retrieval-Augmented Text Generation in Large Language Models. arXiv:2404.10981.

[10] Yunfan Gao, Yun Xiong, Xinyu Gao. (2024). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997.

[11] Saroj Mishra, Suman Niroula, Umesh Yadav. (2024). SoK: Agentic Retrieval-Augmented Generation (RAG): Taxonomy, Architectures, Evaluation, and Research Directions. arXiv:2603.07379.

[12] Weihang Su, Yichen Tang, Qingyao Ai. (2024). Parametric Retrieval Augmented Generation. arXiv:2501.15915.

[13] Shangbin Feng, Weijia Shi, Yuyang Bai. (2023). KNOWLEDGE CARD: FILLING LLMS' KNOWLEDGE GAPS WITH PLUG-IN SPECIALIZED LANGUAGE MODELS. arXiv:2305.09955.

[14] Peng Xu, Wei Ping, Xianchao Wu. (2023). Retrieval Meets Long Context Large Language Models. arXiv:2310.03025.

[15] Fei Wang, Xingchen Wan, Ruoxi Sun. (2024). ASTUTE RAG: Overcoming Imperfect Retrieval Augmentation and Knowledge Conflicts for Large Language Models. arXiv:2410.07176.

[16] Chunyang Li, Junwei Zhang, Anda Cheng. (2024). CPA-RAG:Covert Poisoning Attacks on Retrieval-Augmented Generation in Large Language Models. arXiv:2505.19864.

[17] Bang An, Shiyue Zhang, Mark Dredze. (2024). RAG LLMs are Not Safer: A Safety Analysis of Retrieval-Augmented Generation for Large Language Models. arXiv:2504.18041.

[18] Zackary Rackauckas. (2024). RAG-FUSION: A NEW TAKE ON RETRIEVAL-AUGMENTED GENERATION. arXiv:2402.03367.

[19] Kun Ran, Shuoqi Sun, Khoi Nguyen Dinh Anh. (2025). RMIT–ADM+S at the SIGIR 2025 LiveRAG Challenge: G-RAG: Generation-Retrieval-Augmented Generation. arXiv:2506.14516.

[20] Alex Laitenberger, Christopher D. Manning, Nelson F. Liu. (2024). Stronger Baselines for Retrieval-Augmented Generation with Long-Context Language Models. arXiv:2506.03989.

[21] Uday Allu, Sonu Kedia, Tanmay Odapally. (2026). WEB RETRIEVAL-AWARE CHUNKING (W-RAC) FOR EFFICIENT AND COST-EFFECTIVE RETRIEVAL-AUGMENTED GENERATION SYSTEMS. arXiv:2604.04936.

[22] Rui Yu, Tianyi Wang, Ruixia Liu. (2018). PT-RAG: Structure-Fidelity Retrieval-Augmented Generation for Academic Papers. arXiv:2602.13647.

[23] Yusheng Mu, Qingting Xu, Author Three. (2024). Shall We Pretrain Autoregressive Language Models with Retrieval? A Comprehensive Study. arXiv:2304.06762.

[24] Rubin Wei, Jiaqi Cao, Jiarui Wang. (2024). MLP Memory: Language Modeling with Retriever-pretrained External Memory. arXiv:2508.01832.

[25] Phoebe Klett, Thomas Ahle. (2024). Extended Mind Transformers. arXiv:2406.02332.

[26] Weijia Shi, Sewon Min, Michihiro Yasunaga. (2023). REPLUG: Retrieval-Augmented Black-Box Language Models. arXiv:2301.12652.

[27] Tingjia Shen, Chuan Qin, Hao Wang. (2024). GenKI: Enhancing Open-Domain Question Answering with Knowledge Integration and Controllable Generation in Large Language Models. arXiv:2505.19660.

[28] Chengkai Huang, Yu Xia, Rui Wang. (2024). Embedding-Informed Adaptive Retrieval-Augmented Generation of Large Language Models. arXiv:2404.03514.

[29] Xinran Zhao, Hongming Zhang, Xiaoman Pan. (2023). Thrust: Adaptively Propels Large Language Models with External Knowledge. arXiv:2307.10442.

[30] Tiziano Labruna, Jon Ander Campos, Gorka Azkune. (2024). When to Retrieve: Teaching LLMs to Utilize Information Retrieval Effectively. arXiv:2404.19705.

[31] Junyi Li, Tianyi Tang, Wayne Xin Zhao. (2023). The Web Can Be Your Oyster for Improving Large Language Models. arXiv:2305.10998.

[32] Yongjie Wang, Yue Yu, Kaisong Song. (2025). When Retrieval Succeeds and Fails: Rethinking Retrieval-Augmented Generation for LLMs. arXiv:2510.09106.

[33] Zhihong Shao, Yeyun Gong, Yelong Shen. (2023). Enhancing Retrieval-Augmented Large Language Models with Iterative Retrieval-Generation Synergy. arXiv:2305.15294.

[34] Tian Yu, Shaolei Zhang, Yang Feng. (2024). AUTO-RAG: AUTONOMOUS RETRIEVAL-AUGMENTED GENERATION FOR LARGE LANGUAGE MODELS. arXiv:2411.19443.

[35] Shailja Gupta, Rajesh Ranjan, Surya Narayan Singh. (2024). A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions. arXiv:2410.12837.

[36] Paul J. L. Ammann, Jonas Golde, Alan Akbik. (2024). Question Decomposition for Retrieval-Augmented Generation. arXiv:2507.00355.

[37] Prakhar Verma, Sukruta Prakash Midigeshi, Gaurav Sinha. (2024). Plan*RAG: Efficient Test-Time Planning for Retrieval Augmented Generation. arXiv:2410.20753.

[38] Dun Yuan, Hao Zhou, Di Wu. (2024). Enhancing Large Language Models (LLMs) for Telecommunications using Knowledge Graphs and Retrieval-Augmented Generation. arXiv:2503.24245.

[39] Dun Yuan, Hao Zhou, Xue Liu. (2024). Enhancing Large Language Models (LLMs) for Telecom using Dynamic Knowledge Graphs and Explainable Retrieval-Augmented Generation. arXiv:2602.17529.

[40] Kartik Sharma, Peeyush Kumar, Yunqing Li. (2024). OG-RAG: ONTOLOGY-GROUNDED RETRIEVAL-AUGMENTED GENERATION FOR LARGE LANGUAGE MODELS. arXiv:2412.15235.

[41] Sai Munikoti, Anurag Acharya, Sridevi Wagle. (2024). ATLANTIC: Structure-Aware Retrieval-Augmented Language Model for Interdisciplinary Science. arXiv:2311.12289.

[42] Qinggang Zhang, Shengyuan Chen, Yuanchen Bei. (2024). A Survey of Graph Retrieval-Augmented Generation for Customized Large Language Models. arXiv:2501.13958.

[43] Qiaoyu Tang, Jiawei Chen, Zhuoqun Li. (2024). Self-Retrieval: End-to-End Information Retrieval with One Large Language Model. arXiv:2403.00801.

[44] Ye Wang, Xinrun Xu, Zhiming Ding. (2024). MindRef: Mimicking Human Memory for Hierarchical Reference Retrieval with Fine-Grained Location Awareness. arXiv:2402.17010.

[45] Xiaoxi Li, Zhicheng Dou, Yujia Zhou. (2024). CorpusLM: Towards a Unified Language Model on Corpus for Knowledge-Intensive Tasks. arXiv:2402.01176.

[46] Hyunji Lee, Jaeyoung Kim, Hoyeon Chang. (2023). Nonparametric Decoding for Generative Retrieval. arXiv:2210.02068.

[47] Zhan Peng Lee, Andre Lin, Calvin Tan. (2025). Finetune-RAG: Fine-Tuning Language Models to Resist Hallucination in Retrieval-Augmented Generation. arXiv:2505.10792.

[48] Scott Barnett, Zac Brannelly, Stefanus Kurniawan. (2024). FINE-TUNING OR FINE-FAILING? DEBUNKING PERFORMANCE MYTHS IN LARGE LANGUAGE MODELS. arXiv:2406.11201.

[49] Shaohan Wang, Licheng Zhang, Zheren Fu. (2024). CL-RAG: Bridging the Gap in Retrieval-Augmented Generation with Curriculum Learning. arXiv:2505.10493.

[50] Weijie Liu, Zecheng Tang, Juntao Li. (2024). MemLong: Memory-Augmented Retrieval for Long Text Modeling. arXiv:2408.16967.

[51] Xiaochen Zhao, Kaikai Wang, Xiaowen Zhang. (2025). HyMem: Hybrid Memory Architecture with Dynamic Retrieval Scheduling. arXiv:2602.13933.

[52] Zhanyu Shen, Sijie Cheng, Zhicheng Guo. (2025). AnchorMem: Anchored Facts with Associative Contexts for Building Memory in Large Language Models. arXiv:2604.17377.

[53] Rana Salama, Jason Cai, Michelle Yuan. (2025). MemInsight: Autonomous Memory Augmentation for LLM Agents. arXiv:2503.21760.

[54] Shuo Ji, Yibo Li, Bryan Hooi. (2025). Memory is Reconstructed, Not Retrieved: Graph Memory for LLM Agents. arXiv:2606.06036.

[55] Jingwei Sun, Jianing Zhu, Jiangchao Yao. (2025). Rethinking How to Remember: Beyond Atomic Facts in Lifelong LLM Agent Memory. arXiv:2605.19952.

[56] Xingchen Xiao, Heyan Huang, Runheng Liu. (2025). MASS-RAG: Multi-Agent Synthesis Retrieval-Augmented Generation. arXiv:2604.18509.

[57] Chia-Yuan Chang, Zhimeng Jiang, Vineeth Rakesh. (2024). MAIN-RAG: Multi-Agent Filtering Retrieval-Augmented Generation. arXiv:2501.00332.

[58] Tianpeng Pan, Wenqiang Pu, Licheng Zhao. (2024). Leveraging LLM Agents for Automated Optimization Modeling for SASP Problems: A Graph-RAG based Approach. arXiv:2501.18320.

[59] Haowen Xu, Jinghui Yuan, Anye Zhou. (2024). GenAI-powered Multi-Agent Paradigm for Smart Urban Mobility: Opportunities and Challenges for Integrating Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) with Intelligent Transportation Systems. arXiv:2409.00494.

[60] Peitian Zhang, Shitao Xiao, Zheng Liu. (2018). Retrieve Anything To Augment Large Language Models. arXiv:2310.07554.

[61] Xinze Li, Hanbin Wang, Zhenghao Liu. (2024). Building A Coding Assistant via the Retrieval-Augmented Language Model. arXiv:2410.16229.

[62] Nowfel Mashnoor, Mohammad Akyash, Hadi Kamali. (2024). TimelyHLS: LLM-Based Timing-Aware and Architecture-Specific FPGA HLS Optimization. arXiv:2507.17962.

[63] Yu He Ke, Liyuan Jin, Kabilan Elangovan. (2024). Development and Testing of Retrieval Augmented Generation in Large Language Models - A Case Study Report. arXiv:2402.01733.

[64] Steven Song, Anirudh Subramanyam, Irene Madejski. (2024). LaB-RAG: Label Boosted Retrieval Augmented Generation for Radiology Report Generation. arXiv:2411.16523.

[65] Will E. Thompson, David M. Vidmar, Jessica K. De Freitas. (2024). Large Language Models with Retrieval-Augmented Generation for Zero-Shot Disease Phenotyping. arXiv:2312.06457.

[66] Nicholas Pipitone, Ghita Houir Alami. (2024). LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain. arXiv:2408.10343.

[67] Khandakar Ashrafi Akbar, Md Nahiyan Uddin, Latifur Khan. (2024). Retrieval Augmented Generation-based Large Language Models for Bridging Transportation Cybersecurity Legal Knowledge Gaps. arXiv:2505.18426.

[68] Vishnuprabha V, Daleesha M Viswanathan, Rajesh R. (2024). Augmented Question-guided Retrieval (AQgR) of Indian Case Law with LLM, RAG, and Structured Summaries. arXiv:2508.04710.

[69] Marko Hostnik, Marko Robnik-Sikonja. (2024). Retrieval-augmented code completion for local projects using large language models. arXiv:2408.05026.

[70] Tenghui Huang, Jinbo Wen, Jiawen Kang. (2024). ParaVul: A Parallel Large Language Model and Retrieval-Augmented Framework for Smart Contract Vulnerability Detection. arXiv:2510.17919.

[71] Jiho Shin, Reem Aleithan, Hadi Hemmati. (2024). Retrieval-Augmented Test Generation: How Far Are We?. arXiv:2409.12682.

[72] Vyacheslav Tykhonov, Han Yang, Philipp Mayr. (2024). Chatting with Papers: A Hybrid Approach Using LLMs and Knowledge Graphs. arXiv:2505.11633.

[73] Agada Joseph Oche, Arpan Biswas. (2025). Role of Large Language Models and Retrieval-Augmented Generation for Accelerating Crystalline Material Discovery: A Systematic Review. arXiv:2508.06691.

[74] Yuchen Luo, Jing Xun, Wei Wang. (2024). A DRIVER ADVISORY SYSTEM BASED ON LARGE LANGUAGE MODEL FOR HIGH-SPEED TRAIN. arXiv:2501.07837.

[75] Yixuan Tang, Yi Yang. (2024). MultiHop-RAG: Benchmarking Retrieval-Augmented Generation for Multi-Hop Queries. arXiv:2401.15391.

[76] Jiawei Chen, Hongyu Lin, Xianpei Han. (2024). Benchmarking Large Language Models in Retrieval-Augmented Generation. arXiv:2309.01431.

[77] Udita Patel, Rutu Mulkar, Jay Roberts. (2024). THELMA: Task Based Holistic Evaluation of Large Language Model Applications-RAG Question Answering. arXiv:2505.11626.

[78] Ronak Pradeep, Nandan Thakur, Shivani Upadhyay. (2024). The Great Nugget Recall: Automating Fact Extraction and RAG Evaluation with Large Language Models. arXiv:2504.15068.

[79] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu. (2024). Corrective Retrieval Augmented Generation. arXiv:2401.15884.

[80] Tomoaki Isoda. (2025). SKILL-RAG: Self-Knowledge Induced Learning and Filtering for Retrieval-Augmented Generation. arXiv:2509.20377.

[81] Amin Abolghasemi, Leif Azzopardi, Seyyed Hadi Hashemi. (2024). Evaluation of Attribution Bias in Generator-Aware Retrieval-Augmented Large Language Models. arXiv:2410.12380.

[82] Peizhuo Lv, Mengjie Sun, Hao Wang. (2024). RAG-WM: An Efficient Black-Box Watermarking Approach for Retrieval-Augmented Generation of Large Language Models. arXiv:2501.05249.

[83] Parishad BehnamGhader, Santiago Miret, Siva Reddy. (2023). Can Retriever-Augmented Language Models Reason? The Blame Game Between the Retriever and the Language Model. arXiv:2212.09146.

[84] Zile Qiao, Wei Ye, Yong Jiang. (2024). Supportiveness-based Knowledge Rewriting for Retrieval-augmented Language Modeling. arXiv:2406.08116.

[85] Shi-Qi Yan, Zhen-Hua Ling. (2024). RPO: Retrieval Preference Optimization for Robust Retrieval-Augmented Generation. arXiv:2501.13726.

[86] Zhipeng Xu, Zhenghao Liu, Yukun Yan. (2024). THINKNOTE: Enhancing Knowledge Integration and Utilization of Large Language Models via Constructivist Cognition Modeling. arXiv:2402.13547.

[87] Haiwei Dong, Shuang Xie. (2024). Large Language Models (LLMs): Deployment, Tokenomics and Sustainability. arXiv:2405.17147.

[88] Kaushik Dwivedi, Padmanabh Patanjali Mishra. (2025). AutoRAG-LoRA: Hallucination-Triggered Knowledge Retuning via Lightweight Adapters. arXiv:2507.10586.

[89] Yue Guo, Wei Qiu, Gondy Leroy. (2024). Retrieval Augmentation of Large Language Models for Lay Language Generation. arXiv:2211.03818.

[90] Aoran Gan, Hao Yu, Kai Zhang. (2025). Retrieval Augmented Generation Evaluation in the Era of Large Language Models: A Comprehensive Survey. arXiv:2504.14891.

[91] Abrar Hamed Al Barwani, Abdelaziz Amara Korba, Raja Waseem Anwar. (2024). User-Centric Phishing Detection: A RAG and LLM-Based Approach. arXiv:2601.21261.

[92] Xiaoyang Chen, Ben He, Hongyu Lin. (2024). Spiral of Silence: How is Large Language Model Killing Information Retrieval?—A Case Study on Open Domain Question Answering. arXiv:2404.10496.

[93] Riccardo Terrenzi, Phongsakon Mark Konrad, Tim Lukas Adam. (2024). A Reference Architecture for Agentic Hybrid Retrieval in Dataset Search. arXiv:2604.16394.

[94] Ruochen Zhao, Hailin Chen, Weishi Wang. (2023). Retrieving Multimodal Information for Augmented Generation: A Survey. arXiv:2303.10868.

[95] Zaifu Zhan, Shuang Zhou, Xiaoshan Zhou. (2024). Retrieval-augmented in-context learning for multimodal large language models in disease classification. arXiv:2505.02087.

[96] Alireza Salemi, Hamed Zamani. (2025). Comparing Retrieval-Augmentation and Parameter-Efficient Fine-Tuning for Privacy-Preserving Personalization of Large Language Models. arXiv:2409.09510.

[97] Jincheol Jung, Hongju Jeong, Eui-Nam Huh. (2024). Federated Learning and RAG Integration: A Scalable Approach for Medical Large Language Models. arXiv:2412.13720.

[98] Yuzhe Zhang, Yipeng Zhang, Yidong Gan. (2024). Causal Graph Discovery with Retrieval-Augmented Generation based Large Language Models. arXiv:2402.15301.

[99] Riccardo Pozzi, Matteo Palmonari, Andrea Coletta. (2024). ReFactX: Scalable Reasoning with Reliable Facts via Constrained Generation. arXiv:2508.16983.